

A Fine-Tuned Control-Theoretic Approach for Dynamic Adaptive Streaming Over HTTP

Maria Silvia Ito, Daniel Bezerra, Stenio Fernandes,
Djamel Sadok
Informatics Center
Recife, Pernambuco, Brazil
Email: msi, db, sflf, jamel@cin.ufpe.br

Géza Szabó
Ericsson Research, Traffic Laboratory, Hungary
Email: geza.szabo@ericsson.com

Abstract—Commercial implementations of adaptive streaming systems are either too aggressive or too conservative, not being able to efficiently deliver video content to the client. Because of these issues, a great number of adaptive streaming systems have been proposed in the academia. They employ different strategies, to select the proper video level that the server should send to the client. One trend is the use of control theory, because it guarantees system stability, and enables simulation and tuning of the system prior to its implementation. However, the current proposals that employ control theory do not take full advantage of this technique, because they do not thoroughly analyze the system parameters and their influence on the overall system performance. Therefore, it is possible that they use suboptimal solutions. In this paper, we propose a control-theoretic approach for selecting the video quality in a DASH environment. We use a controller added to a state machine, to improve the QoE and to maximize bandwidth utilization. We also present a comparison of several system settings performances, by changing the controller types, their gains, and the state machine parameters. Experimental results show the effectiveness of our proposed system for buffer time control and video level smoothness, and that it outperforms a current proposal in the academia.

I. INTRODUCTION

Multimedia streaming services are popular in a wide variety of scenarios. This results from the growing marketing and popularity of mobile devices with high storage and computational capacities, high-speed wireless network interfaces, and quality adaptive encoding systems [1]. Indeed, in 2012 video traffic accounted for more than half of global consumer traffic over the Internet, and on-demand video services will continue to grow up to 2016 and beyond [2].

Content providers have the challenge of delivering a seamless multimedia experience while providing optimal Quality of Experience (QoE), taking into account different client network connections and devices. To address this issue, multimedia content delivery needs to be adaptive to a wide range of variables, such as user screen resolution, CPU load, network conditions, and power consumption [3]. If these variables and conditions are not properly addressed, either portions of video content are dropped or bandwidth are wasted [4]. In fact, adaptive streaming presents several advantages over traditional approaches, such as little buffering, fast start time, and good QoE for either good or bad network connections [5]. To this end, HTTP adaptive streaming has emerged as an efficient solution, due to its simplicity and ease of working through firewalls and Network Address Translation (NAT) boxes [6].

With adaptive streaming, a given piece of video content is encoded into several versions, at different bitrates. Each version is called a video level or video quality, and they are split into several chunks or segments, containing seconds of video. By using HTTP, clients send HTTP GET requests to download the segments from the server. As network conditions change, clients must dynamically request segments to obtain different video levels, to match their current available bandwidth [6].

Aiming at standardizing adaptive streaming systems, the Moving Picture Experts Group (MPEG) working group has proposed Dynamic Adaptive Streaming over HTTP (DASH). The DASH standard describes delivery formats to allow interoperability between solutions [7]. DASH has intentionally not specified the client behavior for content fetching, rate adaptation heuristics, and video playback. This allows media delivery applications to use a wide variety of factors in their adaptation techniques [8].

There is a wide range of research proposals in the literature regarding video adaptation techniques, including general controllers, which employ heuristics in their video level selection. There are also several proposals with a strong mathematical basis and, therefore, present an analytical study of the proposed system. Particularly, control theory has been widely used in this field, because it allows the designer to mathematically analyze [9] and simulate the system behavior, guaranteeing system stability [2] and its tuning prior to implementation.

In this paper, we present a control-theoretic based DASH system, using a control system added to a state machine. We add the state machine to the control system to provide seamless video to the client with the highest possible quality, and to prevent the buffer from overflow and underflow. We use average download rate instead of bandwidth estimation, because it is a simpler and more accurate way to observe the network conditions. Also, because the control system and the state machine interfere on each other's behavior, it is not possible to make a simple selection of their parameters. Therefore, we can not select such parameters separately, because it may lead to suboptimal solutions. We study the whole system's performance under several parameter selections.

The remainder of the paper is organized as follows. Section II presents an overview of the proposals that utilize control theory to adaptive streaming systems. Section III presents our proposed system, and its stability analysis is provided in Section IV. In Section V, we present the performance

analysis of our system, compared to an existing proposal in the academia, [6]. In Section VI, we present our concluding remarks.

II. RELATED WORK

There are several commercial players that employ Adaptive Streaming, such as Netflix, Hulu, Vudu, Akamai, Livestream, and YouTube [10]. The most common architecture is the implementation of adaptive streaming using a controller at the client side, which allows the use of standard HTTP servers and provides scalability [10].

There has been a great interest in the performance evaluation of such commercial players [11], [12], [13], [14], [15]. Overall, they do not deliver the video content efficiently, because they are either too aggressive or too conservative. Because of these issues, a great number of adaptive streaming systems have been proposed [10].

Control theory has gained attention in the video level selection strategies, and has been used in a wide variety of papers [1], [2], [6], [9], [16], [17], [18], [19]. The authors use different approaches, for example the control at the server [1], [9] or at the client [2], [6], [18], [19], among others. For example, Cicco et al. [9] propose Quality Adaptation Controller (QAC), designed for live video streaming. QAC is a controller located at the server side, to mitigate delay effects and simplify player design. Fortuna et al. [1] propose Quality Adaptive Scheduler (QAS), a system for properly distributing the available bandwidth between base and enhancement layers of scalable video. Its main goal is preventing the base layer buffer from having underflow. A controller is located at the server, coupled with a Smith Predictor, to mitigate the effect of disturbances and delay. However, because both proposals implement the control at the server, they may present scalability issues. Also, they do not present an analysis of the control system parameter selection, and do not provide stability analysis, fundamental in the project of a control system.

Zhou et al. [6] propose an approach, located at the client side, for rate adaptation that considers bandwidth and buffer time. The goal is a balance between video quality smoothness and bandwidth utilization. The authors present some remarks and propositions regarding system stability and response characteristics, such as settling time. However, they do not present an analysis on controller type and parameters selection. Also, the system features add high conservativeness to the system, causing it a long time to react to network variations, which may lead to the delivery of unnecessary low video quality to the client, and to low bandwidth utilization. Such conservativeness may also cause playback freezes.

Overall, these papers employ control theory in their designs, however there is not a deep concern regarding the control system itself. The authors do not provide a deep explanation on controller type and its parameters selection. Some of the proposals employ pure control systems, however a control system objective is to pursue a certain buffer time. That is not enough for ensuring video smoothness [18], which is among the main goals of video delivery systems. Other papers employ control systems added to some enhancements, to achieve smooth video delivery. However, they do not consider that such coupling could be improved with tuning the control system and

enhancement parameter selection. In this paper, we present a control-theoretic based DASH system, by using a control system added to a state machine. We present the overall system design, its stability analysis, and evaluate the performance of different system parameters combinations, to select a fine-tuned set of parameters.

III. PROPOSAL

In this section, we describe our DASH system proposal. Our system considers some of the features employed in [6], because of its mathematical definitions. First, we detail the control-theoretic specifications, and then we describe the state machine behavior. Our goal is to: 1) deliver video to the client with high QoE, and 2) maximize the bandwidth utilization. The former is obtained by avoiding playback freeze, delivering high video quality, and providing video smoothness, among others [20]. The latter is achieved with the client requesting high video quality, allowed by the network conditions and client buffer. Because of the varying network conditions, it is not possible to deliver continuous and smooth video with requesting the highest video quality allowed by the network. Therefore, we need to find a trade-off between our mentioned objectives.

In our proposal, displayed in Figure 1, the video level selection system, i.e., adaptation logic, is located at the client. We use a standard HTTP server, which stores the video content at different video qualities. All the video qualities are segmented into chunks with fixed duration, and the client requests the proper video quality to the server via GET requests. The server, then, sends the chunks in different qualities, according to the client requests. The variables in the Figure will be further explained in this Section.

Our system controls the client buffer time and, based on that, on the average download rate and on the system state, it requests other chunks, specifying their video quality. It comprises two modules, namely the control system and the state machine. The control system has buffer time as controlled variable: it aims at keeping the buffer time at a constant value q_0 . Therefore, its main goal is to avoid buffer underflow and overflow: it is not able to control the video level switch frequency, not guaranteeing smooth video delivery. We use buffer time instead of buffer level, because the buffer contains chunks with different video qualities (i.e., different bitrates), therefore it is no longer possible to relate the buffer size to buffered video time [6], [18].

The state machine determines the system aggressiveness, by defining the video level switch frequency. If network condition variation is abruptly followed by a video quality switch, the system is considered *aggressive*. Such feature allows high bandwidth utilization. However, it tends to cause frequent and sometimes unnecessary video level switches, which is annoying to the client. On the other hand, if the system reacts slowly to network condition variation, it is considered *conservative*. It avoids changing the video quality too often, however bandwidth efficiency decreases in this case. Our goal is to obtain a trade-off between the two mentioned aggressiveness profiles.

In a nutshell, the control system calculates the ideal video quality to be requested $l(k)$, considering the buffer time and the

average download rate. We use average download rate ($\bar{D}(k)$), once it is a simpler and more accurate method to observe the network conditions than available bandwidth estimation. The ideal video quality is, then, fed to the state machine along with the buffered video time. The state machine considers the received variables' values and decides whether or not to switch the requested video level, requesting $l_c(k)$. In the following subsections, we provide a deeper explanation of these two modules.

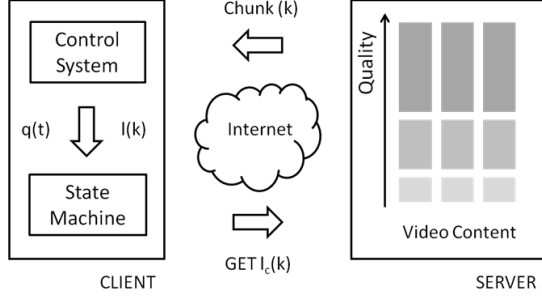


Fig. 1. Proposed architecture

1) *Control system*: The control system uses buffer time as reference variable, therefore, its main goal is to keep the buffer time at a given fixed value q_0 . After the download of a chunk k , it calculates the player buffer time and average download of the chunk. It also considers the chunk durations, which is constant in our system. With these values, it calculates the desired video quality to be requested for the next chunk. Such value is, then, sent to the state machine, which selects the video quality that is actually requested.

Let's consider a video content, stored in a server that contains n quality levels of the given video. Let the set of quality levels be $\mathcal{L} = \{l_1, l_2, \dots, l_n\}$, and the selected video level for a chunk k be $l(k) \in \mathcal{L}$. Chunk k is downloaded at an average download rate of $\bar{D}(k)$, and its extraction rate, i.e., the video time that is extracted by the player from its buffer, is $d_e(k)$. In our system, the duration of all the chunks is x . Table I shows a set of definitions and symbols that will be used in the remainder of the paper.

Definitions:	
Chunk download start [s]	$t_s(k)$
Chunk download end [s]	$t_e(k)$
Chunk size [kb]	$S(k)$
Chunk duration [s]	x
Chunk bitrate (control system) [kbps]	$l(k)$
Buffer time at time t	$q(t)$
Video rate selection (before quantization) [kbps]	$l'(k)$
Decoder extraction for a given chunk [dimensionless]	$d_e(k)$
Average download rate for chunk k [kbps]	$\bar{D}(k)$

TABLE I. SYSTEM DEFINITIONS

For the sake of system model simplification, we assume that the client-server session is in greedy mode, i.e., that $t_e(k) = t_s(k+1)$. Also, we assume $\bar{d}_e(k) > 0$ (considering that there are no playback freezes) and $\bar{D}(k) > 0$ (considering a network that is not too congested). All the assumptions are reasonable and applicable in a real world scenario.

A chunk size can be obtained as:

$$(t_e(k) - t_s(k)) \times \bar{D}(k) = S(k) = l(k) \times x \quad [kbps]$$

After the download of a chunk, the buffer time is:

$$\begin{aligned} q(t_e(k)) &= q(t_s(k)) + x - (t_e(k) - t_s(k)) \times d_e(k) \quad [s] \\ &= q(t_s(k)) + x - \frac{l(k) \times x}{\bar{D}(k)} \times d_e(k) \quad [s] \end{aligned}$$

In order to evenly distribute the downloaded chunk time over its download interval, we apply fluid approximation, by calculating the derivative of buffer time [2], [18], [19]:

$$\begin{aligned} q'(t) &\approx \frac{q(t_e(k)) - q(t_s(k))}{t_e(k) - t_s(k)} \\ &= \frac{q(t_s(k)) + x - \frac{l(k) \times x}{\bar{D}(k)} \times d_e(k) - q(t_s(k))}{\frac{l(k) \times x}{\bar{D}(k)}} \\ &= \frac{\bar{D}(k)}{l(k)} - d_e(k) \end{aligned}$$

We intend to follow a certain buffer time reference q_0 . We can obtain such value by utilizing q_{min} and q_{max} as thresholds.

For the lower threshold,

$$q(t_e(k)) \geq q_{min} \quad [s]$$

$$q(t_s(k)) + x - \frac{l(k) \times x}{\bar{D}(k)} \times d_e(k) \geq q_{min} \quad [s]$$

$$l(k) \leq \frac{\bar{D}(k)}{x \times d_e(k)} \times (q(t_s(k)) - q_{min}) + \frac{\bar{D}(k)}{d_e(k)} \quad [kbps]$$

For the upper threshold,

$$q(t_e(k)) \leq q_{max} \quad [s]$$

$$q(t_s(k)) + x - \frac{l(k) \times x}{\bar{D}(k)} \times d_e(k) \leq q_{max} \quad [s]$$

$$l(k) \geq \frac{\bar{D}(k)}{x \times d_e(k)} \times (q(t_s(k)) - q_{max}) + \frac{\bar{D}(k)}{d_e(k)} \quad [kbps]$$

Then, we make $q_{max} \rightarrow q_0$ and $q_{min} \rightarrow q_0$, and obtain:

- $l(k) = \frac{\bar{D}(k)}{x \times d_e(k)} \times (q(t) - q_0) + \frac{\bar{D}(k)}{d_e(k)} \quad [kbps]$
- $q'(t) = \frac{\bar{D}(k)}{l(k)} - d_e(k)$

Our proposed block diagram, in the time domain, is displayed in Figure 2. Where Q is the quantization function and R is the reciprocal function, i.e., $R(f(k)) = \frac{1}{f(k)}$. It is a nonlinear system, not desirable since it adds a great amount of complexity to the analysis.

Therefore, we employ the linearization technique [21] to the system, around an operating point $q'(t) = 0 \Rightarrow l_0 = \frac{\bar{D}(k)}{d_e(k)}$. Then, we obtain the linearized state and output:

- $\delta l = \frac{\bar{D}(k)}{x \times d_e(k)} \times \delta q$

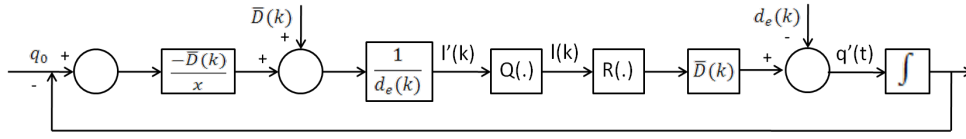


Fig. 2. Block diagram, in the time domain

- $\delta q' = -\frac{(d_e(k))^2}{D(k)} \times \delta l$

Where $\delta l = l(k) - l_0$ and $\delta q' = q'(t) - q'_0$. Finally, we can derive the closed-loop transfer function of the system, by applying the Laplace transform and manipulating the equation system. In the equation, $s \in \mathbb{C}$ denotes the Laplace variable:

$$H_0(s) = \frac{d_e(k)}{s \times s + d_e(k)}$$

After that, we insert a controller, $G_c(s)$, to the system. Figure 3 shows the linearized block diagram with the controller, in the frequency domain. Therefore, the final system transfer function is:

$$H_c(s) = \frac{G_c(s) \times d_e(k)}{s \times s + d_e(k) \times G_c(s)}$$

There are several controller types in the literature. In this paper, we are going to focus on the PID controllers, which may present proportional, derivative and integral gains (k_p , k_d and k_i , respectively). Such gains determine the influence of present, future and past events, respectively [21]. Depending on the gains they present, they may be called P (proportional), PI (proportional-integrative), PD (proportional-derivative), or PID (proportional-integrative-derivative) [22]. There are other controller configurations that employ these gains, however they are out of the scope of this paper.

In the following subsection, we describe the state machine module, which acts to determine how frequent the requested video level will be switched. It has some additional states, that will act to add more conservativeness with buffer time control.

2) *State Machine*: A pure control system is sufficient for buffer time stabilization, however it does not ensure video level smoothness. Therefore, we propose a state machine that allows video level selection smoothness, avoiding, frequent and unnecessary video quality switches. We also add some states that enhance buffer time control, to avoid buffer time overflow or underflow.

The state machine, illustrated in Figure 4, receives the selected video level $l(k)$ and buffer time $q(t)$ from the control system. It also observes previous video level requests $l_c(k)$, and has two buffer safety thresholds, q_{max} and q_{min} . One should observe, from Figure 1 that $l(k)$ is an input to the state machine, sent from the control system. It is different from the requested video level to the server, $l_c(k)$. Consider a current state l_c , the current requested video level:

- If $q(t) < q_{min}$, the requested video rate switches to the lowest level (l_0 state). This state attempts to avoid buffer underflow. However, if the network conditions are too severe, underflow may be unavoidable.

- If $q(t) > q_{max}$, the download starts an idle stage (IDLE state). Such state attempts to avoid buffer overflow and packet drop. The idle stage is designed to extract some video time from the buffer before trying to fill it again.
- If $q_{min} \leq q(t) \leq q_{max}$, two actions may be taken:
 - If $l(k) > l_c(k-1)$ for m chunks, switch up the video level request $l_c(k-1)$ to $l(k)$ (l_+ state). This state adds some level of conservativeness, by avoiding unnecessary switch ups in the requested video level.
 - If $l(k) < l_c(k-1)$ for n chunks, switch down the previous video level request $l_c(k-1)$ to $l(k)$ (l_- state). This state adds some conservativeness, by avoiding unnecessary switch downs in the requested video level. However, because we do not want the buffer to underflow, a careful selection of the parameter n should be made.
- Otherwise, the video request should remain the same (l_c state).

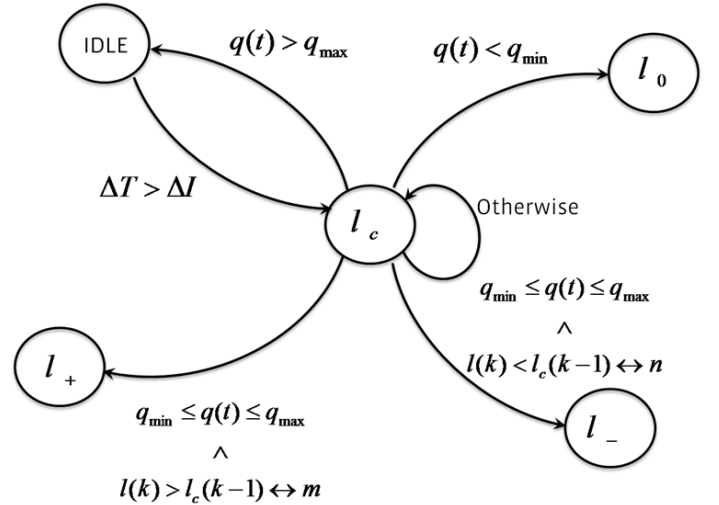


Fig. 4. State machine diagram

Our goal with such conservativeness in the state machine is to avoid unnecessary video quality switches, therefore providing smooth video delivery. One should note that as the video levels switch up or down, the current state is always l_c , because it is the current video level that the client requests. Finally, consider the IDLE state:

- Consider Δ_T the time interval from the change to the IDLE state to the present time. The state machine will remain in this state for $\Delta_I(k)$ seconds. That is, if $\Delta_T > \Delta_I(k)$, the download should restart, returning

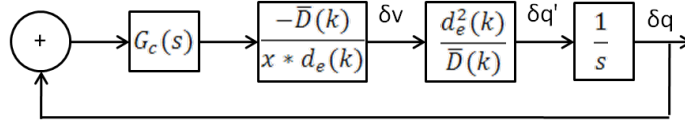


Fig. 3. Block diagram, in the frequency domain, with a controller $G_c(s)$

to the state l_c , downloading video level $l_c(k) = l(k)$.
The threshold $\Delta_I(k)$ is:
$$\Delta_I(k) = q(t_e(k-1)) + x - \frac{l_n \times x}{D(k)} - q_{max}$$

IV. STABILITY ANALYSIS

In this section, we obtain the stability requirements for each control system setting, i.e., for each control system configuration we consider in our analysis. First, we consider the controllers' transfer functions. Then, we derive the system transfer function $H_c(s)$ of every specific case. Finally, we calculate the gain constraints that should be applied to all controllers, considering that the poles of these transfer functions should be located in the left-half s plane to ensure system stability [22].

- P controller

Its transfer function is:

$$G_P(s) = k_p$$

The obtained system transfer function is:

$$H_P(s) = \frac{k_p \times d_e(k)}{x \times s + d_e(k) \times k_p}$$

Therefore, its pole is:

$$s_1 = -\frac{k_p \times d_e(k)}{x}$$

Because the decoder extraction $d_e(k) > 0$ and the chunk duration $x > 0$, the system is stable if we can make sure that:

$$k_p > 0$$

- PD controller

Its transfer function is:

$$G_{PD}(s) = k_p + k_d \times s$$

The obtained system transfer function is:

$$H_{PD}(s) = \frac{(s \times k_d + k_p) \times d_e(k)}{s \times (x + k_d \times d_e(k)) + d_e(k) \times k_p}$$

Its pole is:

$$s_1 = -\frac{k_p \times d_e(k)}{x + k_d \times d_e(k)}$$

The system is stable if:

- $k_p > 0$ and $k_d > \frac{-x}{d_e(k)}$ or
- $k_p < 0$ and $k_d < \frac{-x}{d_e(k)}$

- PI controller

Its transfer function is:

$$G_{PI}(s) = k_p + k_i \times \frac{1}{s}$$

The obtained system transfer function is:

$$H_{PI}(s) = \frac{(s \times k_p + k_i) \times d_e(k)}{s^2 \times x + s \times k_p \times d_e(k) + k_i \times d_e(k)}$$

Its poles are:

$$s_{1,2} = \frac{-d_e(k) \times k_p \pm \sqrt{d_e^2(k) \times k_p^2 - 4 \times x \times k_i \times d_e(k)}}{2 \times x}$$

The system is stable if:

$$k_i > 0$$

- PID controller

Its transfer function is:

$$G_{PID}(s) = k_p + k_d \times s + k_i \times \frac{1}{s}$$

The obtained system transfer function is:

$$H_{PID}(s) = \frac{(s^2 \times k_d + s \times k_p + k_i) \times d_e(k)}{s^2 \times (x + k_d \times d_e(k)) + s \times k_p \times d_e(k) + k_i \times d_e(k)}$$

Its poles are:

$$s_{1,2} = \frac{-d_e(k) \times k_p \pm \sqrt{d_e^2(k) \times k_p^2 - 4 \times (x + k_d \times d_e(k)) \times k_i \times d_e(k)}}{2 \times (x + k_d \times d_e(k))}$$

The system is stable if:

$$k_d > -\frac{x}{d_e(k)}$$

The obtained results are summarized in Table II. One should note that the decoder extraction and chunk duration are positive, i.e., $d_e(k) > 0$ and $x > 0$.

V. EXPERIMENTAL EVALUATION

In this section, we describe the methodology for the control system and state machine parameter selection. Then, we discuss the performance evaluation and compare our approach to the proposal by Zhou et al. [6]. We selected this study for comparison because our system takes into account some of its features, and because of its general applicability and simplicity in implementation.

Our testbed consists of two computers with Ubuntu 12.04 running on both. The server has an Apache module installed, and it stores the content, generated with DASHEncoder [23]. The video we utilize is "Sita sings the blues" [24], long enough for the purpose of our experiments. The video is encoded at five different bitrates $\mathcal{L} = \{300kbps, 700kbps, 1500kbps, 2500kbps, 3500kbps\}$, the same video quality set used in [9]. The client has a modified VLC [25] added to a libdash [26] system. The VLC application is modified so that it uses our adaptation logic, or the adaptation logic presented by Zhou et al. [6], in the client requests. During the experiments, the server runs Netem [27] scripts to constrain the available bandwidth. To check the controllers' reactions, we apply both short-term and long-term step variations, as shown in Figure 7. For the remainder of the Section, we consider the decoder extraction $d_e(k) = 1$, a reasonable assumption, considering we ensure the buffer does not underflow.

A. Parameter Selection

In this subsection, we present the procedure of fine-tuning our proposed system, with the proper selection of its parameters. First, we previously select a group of parameters for both modules. The controller gains and controlled variable selection is accomplished considering the constraints in Section IV and simulating the pure control system. We choose the gains and values that lead to a low rise time and different overshoot

Controller type	P	PD
Controller transfer function	$G_P(s) = k_p$	$G_{PD}(s) = k_p + k_d \times s$
System transfer function	$H_P(s) = \frac{k_p \times d_e(k)}{s \times s + d_e(k) \times k_p}$	$H_{PD}(s) = \frac{(s \times k_d + k_p) \times d_e(k)}{s \times (s + k_d \times d_e(k)) + d_e(k) \times k_p}$
Gain constraint	$k_p > 0$	$k_p > 0$ and $k_d > \frac{-x}{d_e(k)}$ $k_p < 0$ and $k_d < \frac{-x}{d_e(k)}$
Controller type	PI	PID
Controller transfer function	$G_{PI}(s) = k_p + k_i \times \frac{1}{s}$	$G_{PID}(s) = k_p + k_d \times s + k_i \times \frac{1}{s}$
System transfer function	$H_{PI}(s) = \frac{(s \times k_p + k_i) \times d_e(k)}{s^2 \times s + s \times k_p \times d_e(k) + k_i \times d_e(k)}$	$H_{PID}(s) = \frac{(s^2 \times k_d + s \times k_p + k_i) \times d_e(k)}{s^2 \times (s + k_d \times d_e(k)) + s \times k_p \times d_e(k) + k_i \times d_e(k)}$
Gain constraint	$k_i > 0$	$k_d > -\frac{x}{d_e(k)}$

TABLE II. CONTROLLERS, THEIR TRANSFER FUNCTIONS AND CONSTRAINTS

Parameter Settings													
	1	2	3	4	5	6	7	8	9	10	11	12	13
q0	20	20	20	20	20	20	20	40	40	40	40	40	40
kp	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01
ki	0	0	0	0	0	0	0	0	0	0	0	0	0
kd	0.01	0.01	0.01	0	0	0	0	0.01	0.01	0.01	0	0	0
m	10	10	7	10	10	10	7	10	10	10	10	10	10
n	3	4	3	2	3	4	4	2	3	4	2	3	4

TABLE III. S_{13} SETTINGS

values. The state machine parameters are selected with previous experimental measurements. In the control system, the previous controlled variable values are $q_0 = \{20s, 40s\}$ and the controller gains $k_p = \{0.01, 0.1\}$, $k_i = \{0, 0.01, 0.1\}$, and $k_d = \{0, 0.01, 0.1\}$. In the state machine, we set different values for $m = \{1, 3, 7, 10\}$ and $n = \{1, 2, 3, 4\}$.

We perform experiments with all the possible combinations of the parameter values. From that, we analyze the buffer time evolution of all the experiments, and select the measurements that present higher values. More specifically, we select the experiments that present the lowest amount of events with buffer time lower than 10s. We do not perform any filter with high buffer time values, because all the experiments present up to two events with buffer time higher than $q_{max} = 70s$.

Then, we observe the GET request logs, and select the experiments that present the smoothest requests. By smoothest, we mean those experiments that do not change their video quality request in the short-term with the short-term constraints applied by Netem (Figure 7). With that, we reach a set S_{13} , shown in Table III, which contains experiments with similar video quality request logs. Finally, we compare their buffer time evolution, and select the experiments with no buffer underflow after the buffering period. We obtain, then, a set S_3 , which consists of settings 1, 5 and 9 from Table III. It is possible to observe that settings 1 and 9 are very similar, presenting only different q_0 values. Therefore, such variable is not of great influence on the entire system performance. Finally, we select the experiment with the highest buffer time profile, obtaining, therefore, a fine-tuned system setting, S_1 . The setting we consider the best among all the tested possibilities is setting 1, from Table III.

We can observe some patterns in Table III. For example, all k_p values are equal to 0.01 and all k_i values are equal to zero, the lowest gains values. That probably happens because of the oscillation added by the proportional and integral gains as they increase, making the controlled variable $q(t)$ present a higher oscillation and sometimes leading to lower buffer levels. Also, the k_d values oscillate between zero and 0.01. That may be a result of k_d influence on the system when such gain increases. With high k_d values, the response time is longer, which may

cause the buffer to underflow when the available bandwidth severely drops. Therefore, $k_d = 0.1$ may be considered an inappropriate gain value.

We can also note that all m values are higher than n . That is expected, because the state machine should be more aggressive when it comes to decreasing the video level request, to avoid buffer underflow. When it comes to increasing the video level request, on the other hand, it should be more conservative, so that higher video levels are not unnecessarily requested.

From such patterns, we can conclude the importance of previously selecting the system settings with experimental results. That is, if the parameter values are not properly selected, the whole system's performance could be compromised. The experimental comparison of different settings allows us to tune the system's behavior according to our goals (mentioned in Section III), therefore tuning it. Also, we can identify parameters that could be fixed, which is helpful when designing and tuning future control systems, if they are similar to our proposal.

B. Performance comparison

In this subsection, we compare the performance of our proposal, S_1 , to the one presented in [6], which will be named ZHOU in the remainder of the paper.

In Figures 5 and 6, we present the buffer time evolution of S_1 and ZHOU, respectively. We can note the buffer underflow allowed by ZHOU even after the buffer filling period. That is not desired, once the video playback freeze severely impairs the user QoE [20]. ZHOU allows two consecutive buffer underflow events, corresponding to a 10s playback freeze. This leads to the conclusion that buffer time management is more efficient in S_1 , and that the QoE delivered by S_1 to the client is higher, because it does not allow video freeze.

Figures 8 and 9 show the client requests to the server in S_1 and ZHOU, respectively. We can realize, from the Figures, that S_1 requests video with 14 video quality changes during the video session, while ZHOU requests the same video content with 21 video quality switches. Most of these video switches were performed by both control systems because

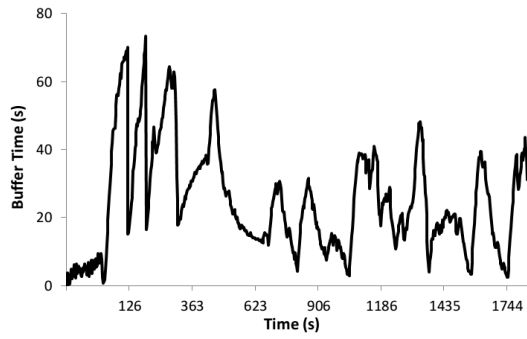
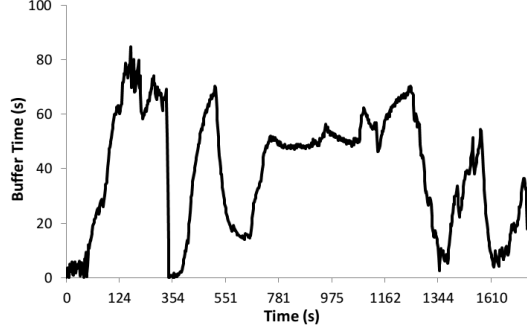
Fig. 5. S_1 buffer time evolution

Fig. 6. ZHOU buffer time evolution

of the reset mechanisms they implement, to avoid buffer underflow. However, even the reset mechanism is not sufficient for avoiding video stall in the ZHOU case. Also, we can note that S_1 bandwidth utilization is higher than ZHOU. More specifically, the S_1 video level request log correspond to a bandwidth utilization of 97%, while ZHOU log correspond to 87% bandwidth utilization. Therefore, the large amount of video quality switches ZHOU performs does not improve the bandwidth utilization, and does not avoid buffer underflows. Considering the buffer time management and the GET request log, one can conclude the delivery of a smoother, non-interrupted video with higher video quality of S_1 when compared to ZHOU. Therefore, according to the correlation between video characteristics (video freeze, quality switch, among others) and QoE [20], the QoE provided by the former is higher, as well as its bandwidth utilization efficiency. Table IV shows a summary of the systems' performance comparison.

	S_1	ZHOU
Buffer underflow events	0	2
Approximate video freeze duration	0	10s
Bandwidth utilization	97%	87%
Number of video quality switches	14	21

TABLE IV. SYSTEMS COMPARISON

The difference in performance of both controllers is due to their strategies. The main disparity between them is the frequency the video level request switches. In ZHOU, such requests are switched when the buffer time is out of the interval 10s, 70s, which may be considered a conservative approach. This leads to a delay in the video level switch, which could cause buffer underflow or overflow when network conditions change severely. The buffer underflow we observe in Figure 6 is probably a cause of that. The delay in shifting up the video level request in Figure 9 happens because the buffer time remains in the interval the controller does not recalculate video

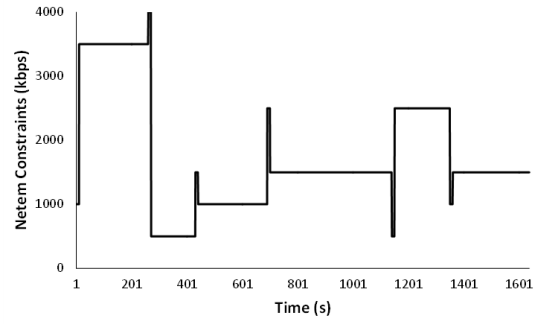


Fig. 7. Netem constraints

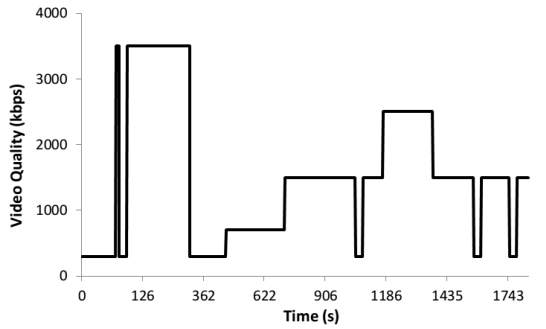
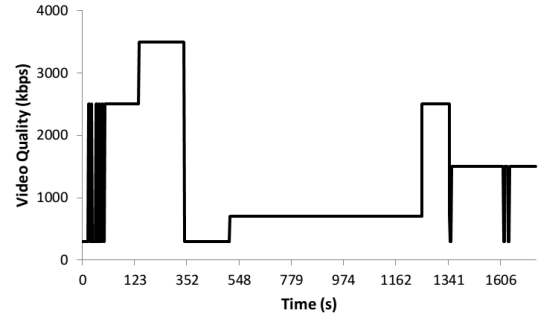
Fig. 8. S_1 client requests

Fig. 9. ZHOU client requests

level requests. Therefore, it causes bandwidth under utilization. Also, when the ZHOU buffer time is out of the mentioned thresholds, it constantly recalculates new video level request. That is not desirable, once it may cause frequent video level switch, especially after a video level reset, seen in Figure 9. In a real scenario, such video level switch would be more evident. However, we should note that ZHOU smooths network condition estimation, allowing the system to smooth the switch video level requests. In fact, such feature is clear in 9, as the video level requests it performs do not follow the short-term bandwidth constraints applied with Netem.

On the other hand, S_1 constantly recalculates the requested video level, not requiring a long time to react to changes in the network conditions and buffer time. Its conservativeness is added with the state machine and the defined numbers previously explained in Section III, m and n . This avoids the sudden switch up after a video level reset. In a real world scenario, the available bandwidth changes more frequently, leading to a higher video level switch frequency. However, the state machine parameters m and n cause delays in shifting the video level, allowing the delivery of a smoother video, while

leveraging the available bandwidth.

VI. CONCLUSION

In this paper, we proposed a control-theoretic approach for selecting the video quality in a DASH environment. Our system consists of a control system and a state machine. We use average download rate instead of employing available bandwidth estimation to determine the network conditions, because it is a simpler and more accurate procedure than the former.

Our proposal's goal is the delivery of smooth video, with no playback freeze, and high bandwidth utilization. Because the control system and the state machine interfere on each other's performance, we proposed a study to select a fine-tuned setting of parameters so that we could deliver video to the client more efficiently and with the highest possible QoE. Experimental results showed the high effectiveness of our proposed system, and that it outperforms a recent architecture in the academia. As future work, we propose the study of our proposal in real-world scenarios, to either confirm our parameter selection, or to make changes to our system. Also, we would like to test our approach using different scenarios, with more clients and mobile devices.

REFERENCES

- [1] R. Fortuna, L. A. Grieco, G. Boggia, and P. Camarda, "Quality adaptive end-to-end packet scheduling to avoid playout interruptions in internet video streaming systems," *J. Syst. Softw.*
- [2] X. Zhu, Z. Li, R. Pan, J. Gahm, and H. Hu, "Fixing multi-client oscillations in http-based adaptive streaming: A control theoretic approach," in *MMSP'13*.
- [3] L. De Cicco and S. Mascolo, "An adaptive video streaming control system: Modeling, validation, and performance evaluation," *Networking, IEEE/ACM Transactions on*.
- [4] Y. Huang, S. Mao, and S. Midkiff, "A control-theoretic approach to rate control for streaming videos," *Multimedia, IEEE Transactions on*.
- [5] P. Xiong, J. Shen, Q. Wang, D. Jayasinghe, J. Li, and C. Pu, "Nbs: A network-bandwidth-aware streaming version switcher for mobile streaming applications under fuzzy logic control," in *MS'12*.
- [6] C. Zhou, C.-W. Lin, X. Zhang, and Z. Guo, "Buffer-based smooth rate adaptation for dynamic http streaming," in *APSIPA'13*.
- [7] T. Stockhammer and I. Sodagar, "Mpeg dash: The enabler standard for video delivery over the internet," *SMPTE Motion Imaging Journal*.
- [8] I. Sodagar, "The mpeg-dash standard for multimedia streaming over the internet," *MultiMedia, IEEE*.
- [9] L. De Cicco, S. Mascolo, and V. Palmisano, "Feedback control for adaptive live video streaming," in *MMSys '11*.
- [10] L. De Cicco, V. Caldaralo, V. Palmisano, and S. Mascolo, "Elastic: A client-side controller for dynamic adaptive streaming over http (dash)," in *Packet Video Workshop (PV), 2013 20th International*.
- [11] T.-Y. Huang, N. Handigol, B. Heller, N. McKeown, and R. Johari, "Confused, timid, and unstable: Picking a video streaming rate is hard," in *IMC '12*.
- [12] S. Akhshabi, L. Anantkrishnan, A. C. Begen, and C. Dovrolis, "What happens when http adaptive streaming players compete for bandwidth?" in *NOSSDAV '12*.
- [13] T. Kupka, P. Halvorsen, and C. Griwodz, "Performance of on-off traffic stemming from live adaptive segmented http video streaming," in *LCN'12*.
- [14] M. Ito, R. Antonello, D. Sadok, and S. Fernandes, "Network level characterization of adaptive streaming over http applications," in *ISCC'14*.
- [15] S. Akhshabi, A. C. Begen, and C. Dovrolis, "An experimental evaluation of rate-adaptation algorithms in adaptive streaming over http," in *MMSys '11*.
- [16] C. Zhou, X. Zhang, L. Huo, and Z. Guo, "A control-theoretic approach to rate adaptation for dynamic http streaming," in *Visual Communications and Image Processing (VCIP), 2012 IEEE*.
- [17] C. Zhou, X. Zhang, and Z. Guo, "A control theory based rate adaption scheme for dash over multiple servers," in *Visual Communications and Image Processing (VCIP), 2013*.
- [18] G. Tian and Y. Liu, "Towards agile and smooth video adaptation in dynamic http streaming," in *CoNEXT '12*.
- [19] C. Zhou, C.-W. Lin, X. Zhang, and Z. Guo, "A control-theoretic approach to rate adaption for dash over multiple content distribution servers," *Circuits and Systems for Video Technology, IEEE Transactions on*, 2014.
- [20] M. Seufert, S. Egger, M. Slanina, T. Zinner, T. Hobfeld, and P. Tran-Gia, "A survey on quality of experience of http adaptive streaming," *Communications Surveys Tutorials, IEEE*, vol. 17, no. 1, pp. 469–492, Firstquarter 2015.
- [21] K. J. Aström and R. M. Murray, *Feedback systems: an introduction for scientists and engineers*. Princeton university press, 2010.
- [22] O. Katsuhiko, *Modern control engineering*.
- [23] S. Lederer, C. Müller, and C. Timmerer, "Dynamic adaptive streaming over http dataset," in *MMSys '12*.
- [24] N. Paley, "Sita sings the blues," <http://sitasingstheblues.com/>, 2009, [Online; accessed 19-December-2014].
- [25] T. V. Organization, "Vlc media player."
- [26] Bitmovin, "libdash."
- [27] T. L. Foundation, "Netem."